Web Engineering 1 | WE1

Summary

CONTENTS

1. HyperText Markup Language (HTML) 3
   - 1.1. Content categories 3
   - 1.2. Semantic elements 4
     - 1.2.1. Webpage structure 5
     - 1.2.2. h* 5
     - 1.2.3. article 5
     - 1.2.4. section 5
     - 1.2.5. img 5
   - 1.3. Void elements 5
2. Cascading Style Sheets (CSS) 6
   - 2.1. Selectors 6
     - 2.1.1. Combinators 6
     - 2.1.2. Specificity 7
   - 2.2. Box-Model 8
   - 2.3. Display 8
     - 2.3.1. Flexbox 8
3. JavaScript (JS) 8
   - 3.1. Atomic language constructs 9
     - 3.1.1. Values and types 9
     - 3.1.2. Functions 9
     - 3.1.3. Objects 9
     - 3.1.4. this 9
     - 3.1.5. ES Modules 9
   - 3.2. The benefits of weak typing /s 10
     - 3.2.1. Equality 10
     - 3.2.2. Operators 10
     - 3.2.3. Numbers 10
     - 3.2.4. Booleans 11
4. Document Object Model (DOM) 11
   - 4.1. Lifecycle 11
   - 4.2. DOM-Tree 11
     - 4.2.1. Nodes 11
   - 4.3. Actions 12
     - 4.3.1. Selection 12
     - 4.3.2. Navigation 12
     - 4.3.3. Manipulation 12
     - 4.3.4. Creation 13
   - 4.4. Events 14
     - 4.4.1. Event-Bubbling 14
5. Clean Code 14
   - 5.1. Naming 14

5.2. Code Quality **15**
5.3. js Styleguide (AirBnB) **15**
5.4. MVC **15**
6. HyperText Transfer Protocol (HTTP) **16**
6.1. URL Scheme **16**
6.2. Request **16**
6.3. Response **17**
6.3.1. Status codes 17
6.3.2. Content-Type 17
6.3.3. Host 18
7. Asynchronous JavaScript And XML (AJAX) **18**
7.1. Async js **18**
7.2. Fetch **19**

WARN:
This document is still under development and will be completed around 1.2.2027

1. HYPERTEXT MARKUP LANGUAGE (HTML)

Structures the contents of a document, defines **what** is content and not **how** it looks.

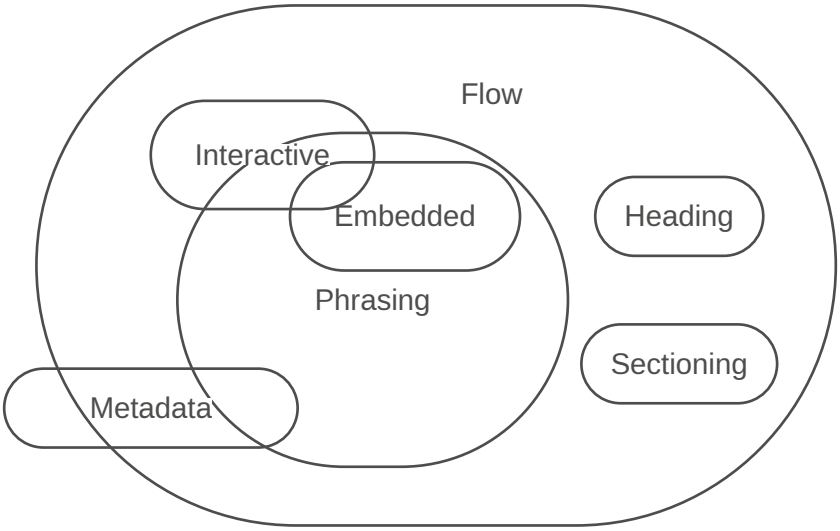
`<head>` contains metadata, `<body>` contains the content.

***Example** 1:*

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <link rel="icon" href="ost-favicon-32x32.png">
  <title>Hello World</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>9/11 was an inside job</p>
</body>
</html>
```

1.1. CONTENT CATEGORIES

Most HTML elements are a member of one or more content categories. The content categories are used to define the content model of elements, in other words, what each element can take as descendants.



Metadata: Elements belonging to the metadata content category modify the presentation or the behavior of the rest of the document, set up links to other documents, or convey other out-of-band information.

Flow: Flow content is a broad category that encompasses most elements that can go inside the `<body>` element, including heading elements, sectioning elements, phrasing elements, embedding elements, interactive elements, and form-related elements.

Sectioning: Creates a section in the current outline defining the scope of `<header>` and `<footer>` elements.

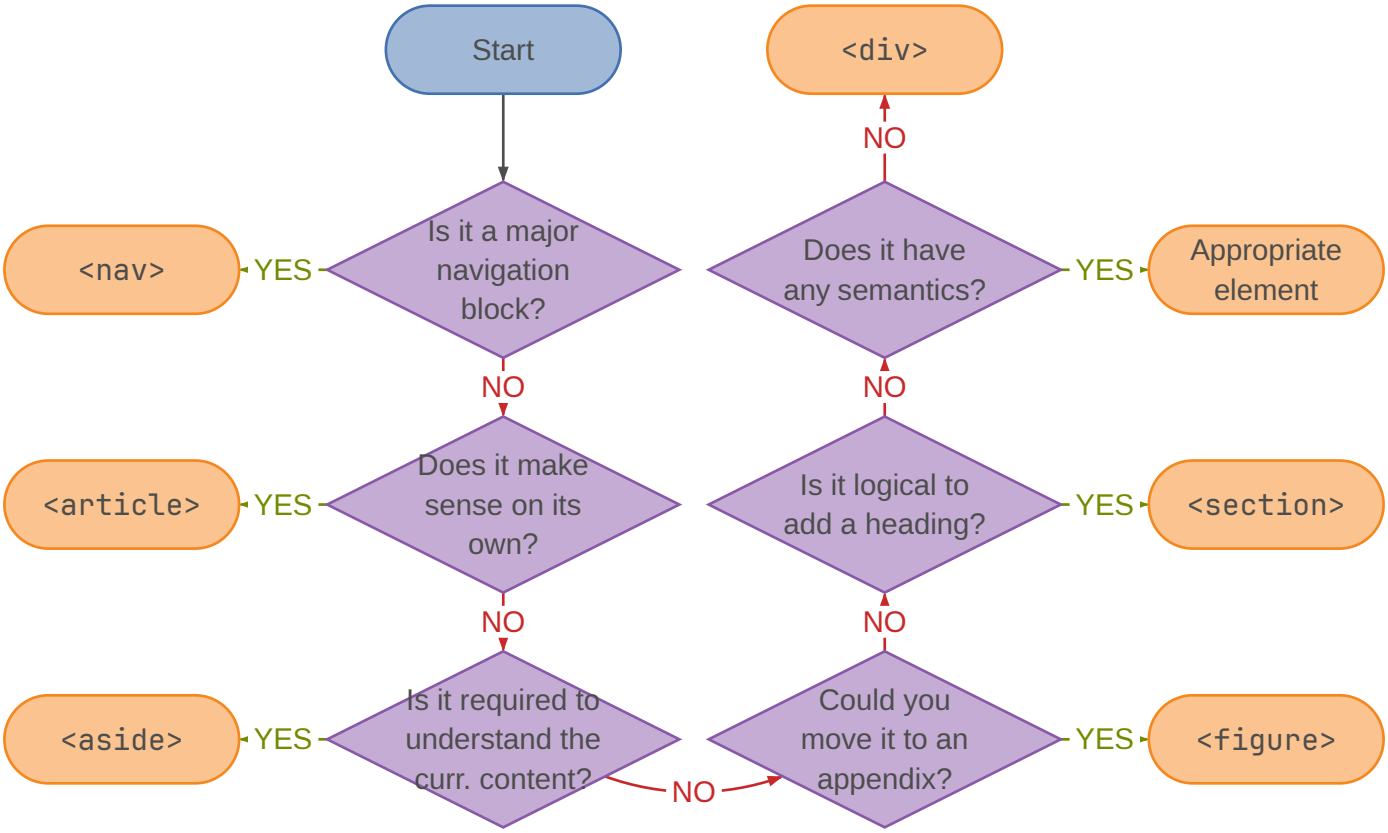
Heading: Defines the title of a section.

Phrasing: Refers to the text and the markup within a document. Sequences of phrasing content make up paragraphs.

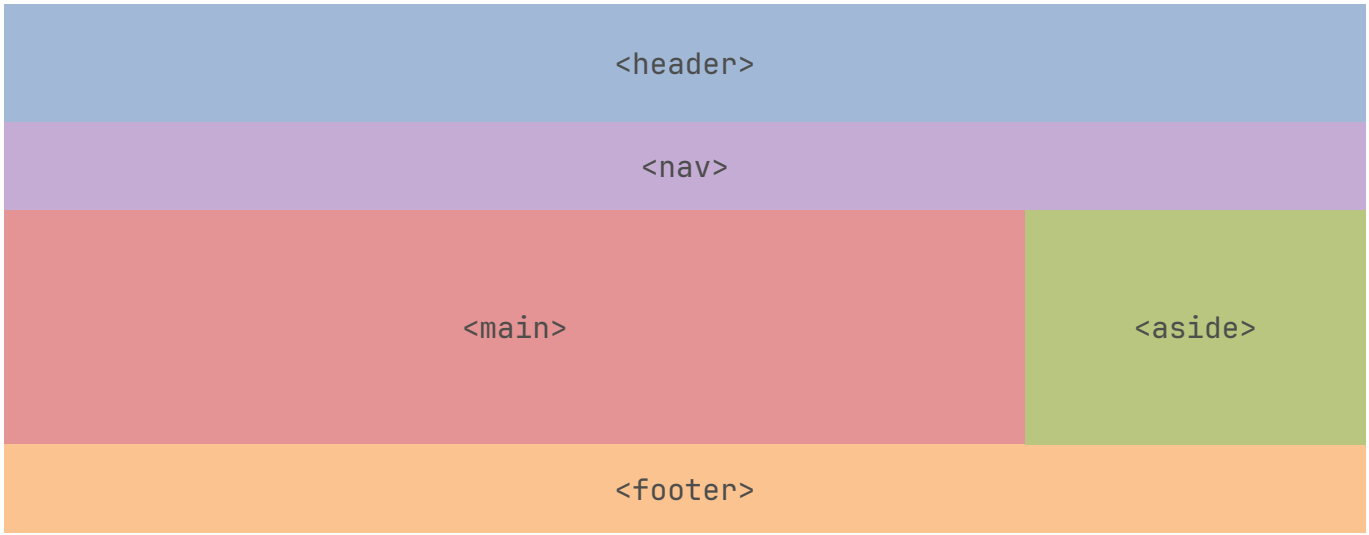
Embedded: Imports another resource or inserts content from another markup language or namespace into the document.

Interactive: Includes elements that are specifically designed for user interaction.

1.2. SEMANTIC ELEMENTS



1.2.1. Webpage structure



1.2.2. h*

<h1> should only appear once on the website.

1.2.3. article

A self-contained part of the document which is independently placed or should be reused.

***Example** 2:*

```
<article>
  <section id="introduction">
    <p>JS was a mistake</p>
  </section>
  <section id="content">
    <!-- very long rant -->
  </section>
  <section id="summary">
    <p>JS was a mistake</p>
  </section>
</article>
```

1.2.4. section

General separation and grouping of content.

1.2.5. img

`alt` attribute's value should provide a clear and concise text replacement for the image's content. It should not describe the presence of the image itself or the file name of the image. The `alt` attribute **must be specified**.

If the image is of decorative nature only, one must still set `alt`, although empty: `alt=""`

1.3. VOID ELEMENTS

A void element is an element in HTML that cannot have any child nodes (i.e., nested elements or text nodes). Void elements only have a start tag; end tags must not be specified for void elements.

`<area> <base> <br> <col> <embed> <hr> <img> <input> <link> <meta> <param> <source> <track> <wbr>`

TODO:
https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements

2. CASCADING STYLE SHEETS (CSS)

A simple mechanism for adding style (e.g., fonts, colors, spacing) to Web documents.

Inline: `<p style="color: red;">Text</p>` (Discouraged)

Inside of HTML: `<head><style> p { color: red; } </style></head>` (For small documents)

External file: `<link rel="stylesheet" href="styles.css">` (Standard)

2.1. SELECTORS

Class: `.classname` reusable for multiple elements

ID: `#idname` for unique, single element

Pseudo-class: Prefixed with a colon (`:`), allow the selection of elements based on state information that is not contained in the document tree.

Pseudo-element: Prefixed with two colons (`::`), represent entities that are not included in HTML. For example, the simple `::marker` selector selects list item bullets, and the compound selector `p::first-line` matches the first line of all `<p>` elements.

Universal: The CSS universal selector (*) matches elements of any type.

Nesting: The CSS & nesting selector explicitly states the relationship between parent and child rules when using CSS nesting.

```
parentRule {
  & childRule { }
}
```

Attribute: The CSS attribute selector matches elements based on the element having a given attribute explicitly set, with options for defining an attribute value or substring value match.

```
a[href*="example"] {
  font-size: 2em;
}
```

2.1.1. Combinators

Descendant combinator: The descendant combinator, denoted with one or more spaces, selects nodes that are descendants of the first element.

Child combinator: The child combinator is more specific than the descendant combinator. Denoted with the greater than character (>), the child combinator selects nodes that are direct children of the first element.

Subsequent-sibling combinator: The subsequent-sibling combinator, denoted with a tilde (~), selects siblings. Given `A ~ B`, all elements matching `B` will be selected if they are preceded by `A`, provided both `A` and `B` share the same parent.

Next-sibling combinator: The next-sibling combinator, denoted by the plus symbol (+), is similar to the subsequent-sibling. However, given `A + B`, it only matches `B` if `B` is **immediately** preceded by `A`, with both sharing the same parent.

Column combinator: There is also a column combinator, denoted by two pipe characters (`||`), which, when supported, selects nodes that belong to a column.

Namespace separator: The namespace separator is another combinator that is generally used in conjunction with the `@namespace` at-rule. This combinator is denoted by a single pipe character (`|`). It enables limiting type selectors and the universal selector to a specific namespace.

2.1.2. Specificity

1) `!important`
2) `style=""` Attribute
3) (1) ID-Selectors
4) (2) Class-Selectors, Pseudoclasses (eg. `:hover`) and Attributes (`[...=...]`)
5) (3) Type-Selectors and Pseudoelements (eg. `::first-line`)

***Example** 3:*

```
div.help > div#nested a:hover::before { }
```

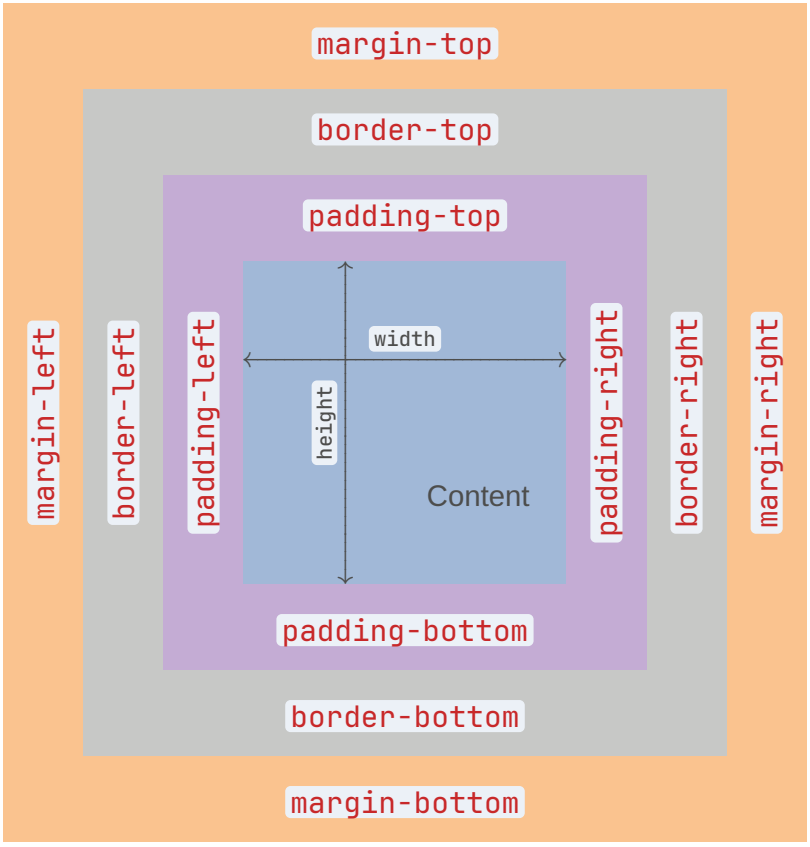
Specificity: 124
1 id, 2 (pseudo-)classes, 4 (pseudo-)elements)

***Example** 4:*

```
<div id="div-1" class="recent significant good-to-go">1</div>
```

```
.recent // overrides others
```

2.2. BOX-MODEL



2.3. DISPLAY

Term	*Definition*
Block (`p, h1, ul`)	Vertical alignment, width fills parent, height from content
Inline (`a, strong`)	Horizontal alignment, width and height from content, `left/right margin/padding` allowed, not `top/bottom`
Inline-Block	Flows like inline, but `width/height margin/padding` all allowed

`display: none;` Element isn't visible and doesn't take any space `visibility: hidden;` Element isn't visible but does use space

2.3.1. Flexbox

flexbox.help

3. JAVASCRIPT (JS)

why did you do this to us, brendan eich

3.1. ATOMIC LANGUAGE CONSTRUCTS

3.1.1. Values and types

JS is dynamically typed (help) and doesn't have constant values. It only has non-reassigneable name-spaces when using `const`. Using `let` you declare a reassigneable variable and using `var` is a crime against humanity.

String interpolation:

```
let answer = 42;
console.log(`The answer is ${answer}`);
```

3.1.2. Functions

Functions are hoisted to the top. A function wihout an explicit return returns `undefined`.

```
function printName(name) {
  console.log(name);
}
```

Lambda functions

```
((a, b) => console.log(a + b == 5))(2, 3)
```

Default parameters

```
function formatAmount(amount, currency = "CHF") {
  return `${currency} ${(amount / 100).toFixed(2)}`;
}
```

3.1.3. Objects

```
let obj = {
  key1: 69,
  key2: "nice",
};

let arr = [ obj, 1, "cursed" ];
```

3.1.4. this

The ***this*** keyword refers to the context where a piece of code, such as a function's body, is supposed to run. Most typically it refers to the object that the method is attached to.

The value of ***this*** in JavaScript depends on how a function is invoked (runtime binding), not how it is defined. which is why the same function can have different ***this*** references in different objects. Arrow functions have no ***this***.

3.1.5. ES Modules

A module is a file with its own scope. You can export functions and values to other modules using `export` and import them using `import`.

```
// validation.js
export function validateInput(input) {
```

```
  // ...
}

// main.js
import { validateInput } from "./validation.js";
```

A module can be used in the browser using `type="module"`. The modules referenced in that module will be recursively loaded as well. Modules behave the same way as `defer`, being executed after the HTML.

```
<head>
  <meta charset="UTF-8">
  <title>Modules</title>
  <script type="module" src="main.js"></script>
</head>
```

3.2. THE BENEFITS OF WEAK TYPING /S

js devs be seething rn

3.2.1. Equality

Equality in js is weird. Double equals (`==`) will perform a type conversion when comparing two things, and will handle `NaN`, `-0`, and `+0` specially to conform to IEEE 754 (so `NaN !== NaN`, and `-0 == +0`). It isn't even transitive. Triple equals (`===`) will do the same comparison as double equals (including the special handling for `NaN`, `-0`, and `+0`) but without type conversion; if the types differ, `false` is returned.

3.2.2. Operators

If one operand of `+` is `typeof x === "string"` then it will convert the other operand into a string as well. Otherwise both operands will coalesce into `Number`.

Other operators (`-, *, /, %`) will convert both operands into `Number`.

Evaluation happens in the order of PEMDAS and from left to right.

3.2.3. Numbers

Expression	*Value*	*Description*
`Number("1.2ab")`	`NaN`	The whole string must be a number
`Number("")`	`0`	Empty string is converted to `0`
`Number("1,5")`	`NaN`	A comma is not a decimal point
`parseFloat("1.2ab")`	`1.2`	Float is read up until invalid input
`parseInt("1.2ab")`	`1`	Int is read up until invalid input

3.2.3.1. Not a Number (NaN)

```
const div0 = 0 / 0;
div0;                  // NaN
typeof div0;           // number
div0 === NaN;          // false
Number.isNaN(div0);    // true
```

```
const inf = 3 / 0;     // Infinity
typeof inf;            // number
```

3.2.3.2. Floating point

`number` is a 64-Bit IEEE 754 floating point number and thus has the usual floating point shenanigans.

Comparing floats can be done in the following ways

```
0.1 + 0.2;                                    // 0.30000000000000004
0.1 + 0.2 === 0.3;                            // false
0.3;                                          // 0.3
const sum = Math.round(0.1 * 100) + Math.round(0.2 * 100);
sum;                                          // 30
sum === Math.round(0.3 * 100)                 // true
Math.abs(0.1 + 0.2 - 0.3) < Number.EPSILON    // true
```

3.2.4. Booleans

`""`, `NaN`, `null`, `undefined` and `0` are all falsy. You shouldn't compare using implicit boolean conversion or using weak equality (`==`). Truthy values are e.g. `"0"`, `"false"`, `[]`, and `{}` which might all lead to bugs. Use strict equality `===`.

4. DOCUMENT OBJECT MODEL (DOM)

The DOM (Document Object Model) is an API that represents an HTML document as a tree structure, where each part (like elements and text) is a node.

4.1. LIFECYCLE

DOMContentLoaded → load → beforeunload → unload

The `DOMContentLoaded` event is fired when the HTML document has been fully loaded and parsed, without waiting for stylesheets, images, and subframes to finish loading.

The `load` event is fired when the whole page, including all dependent resources such as stylesheets and images, has loaded.

The `beforeunload` event is fired when the window, the document, and its resources are about to be unloaded.

The `unload` event on `window` triggers when the user is finally leaving, in the handler we can only do simple things that do not involve delays or asking a user. Because of that limitation, it's rarely used.

4.2. DOM-TREE

The HTML Document is represented as a tree in the DOM. It offers an API for HTML Documents which provide functions to query the structure and its nodes, CRUD'ing them, registering event-handlers etc.

4.2.1. Nodes

The DOM Node interface is an abstract base class upon which many other DOM API objects are based, thus letting those object types be used similarly and often interchangeably. All objects that implement Node functionality are based on one of its subclasses. Most notable are `Document`, `Element`, and `DocumentFragment`.

In addition, every kind of DOM node is represented by an interface based on Node. These include Attr, CharacterData (which `Text`, `Comment`, `CDATASection` and `ProcessingInstruction` are all based on), and `DocumentType`.

4.3. ACTIONS

4.3.1. Selection

```
document.querySelector("h2");     // returns first matching element
document.querySelectorAll("h2");  // returns all matching elements
```

4.3.2. Navigation

```
node.parentElement
node.childNodes
node.children
node.firstChild
node.firstElementChild
node.nextSibling
node.nextElementSibling
```

4.3.3. Manipulation

4.3.3.1. CSS classList

***Example** 5:*

```
element.classList     // a DOMTokenList

// use the classList API to remove and add classes
element.classList.remove("foo");
element.classList.add("another-class");

// if visible is set remove it, otherwise add it
element.classList.toggle("visible");

// add/remove visible, depending on test conditional, i less than 10
element.classList.toggle("visible", i < 10);

// add or remove multiple classes
element.classList.add("foo", "bar", "baz");
element.classList.remove("foo", "bar", "baz");

// replace class "foo" with class "bar"
element.classList.replace("foo", "bar");
```

4.3.3.2. Contents

`innerHTML`, `innerText`, and `textContent` are properties of the DOM Nodes. They allow you to read and update the content of HTML elements.

innerHTML: reads both the HTML markup and the text content of the element. This means when you use it to set the content of elements, you can include HTML tags, and the browser will render them correctly.

innerText: returns the text as it appears on screen. It ignores HTML tags

textContent: also ignores all HTML tags and returns only the text. While `innerText` reads text as it is rendered on screen, textContent reads text as it is in the markup. It also returns all text, whether it's rendered on screen or not

4.3.4. Creation

createElement(): This method of the Document interface creates a new `HTMLElement` that has the specified `localName` (type of node). If `localName` isn't recognized, the method creates an `HTMLUnknownElement`.

createTextNode(): Creates a new Text node. This method can be used to escape HTML characters.

createDocumentFragment(): Creates a new empty `DocumentFragment` into which DOM nodes can be added to build an offscreen DOM tree.

***Example** 6:*

```
const element = document.getElementById("elem");
const fragment = document.createDocumentFragment();
const browsers = ["Firefox", "Chrome", "Opera", "Safari"];

browsers.forEach((browser) ⇒ {
  const li = document.createElement("li");
  li.textContent = browser;
  fragment.appendChild(li);
});

element.appendChild(fragment);
```

4.3.4.1. insertAdjacentHTML

Use this method if you want to provide XSS attack surface.

***Example** 7:*

```
let container = document.querySelector("#container");
let newHTMLCode = "<p>igrok</p>";
container.insertAdjacentHTML("beforeend", newHTMLCode);
```

`position` can be one of the following:

beforebegin: Before the element. Only valid if the element is in the DOM tree and has a parent element.

afterbegin: Just inside the element, before its first child.

beforeend: Just inside the element, after its last child.

afterend: After the element. Only valid if the element is in the DOM tree and has a parent element.

4.4. EVENTS

Objects that can fire events have an `addEventListener()` method.

***Example** 8:*

```
let handleClick = e ⇒ console.log(`Clicked on ${e.target}`)
element.addEventListener("click", handleClick);
element.removeEventListener("click", handleClick);
```

e.preventDefault(): prevents the browser from executing the default behavior of the selected element

e.stopPropagation(): prevent the parent element from accessing the event (stop bubbling)

4.4.1. Event-Bubbling

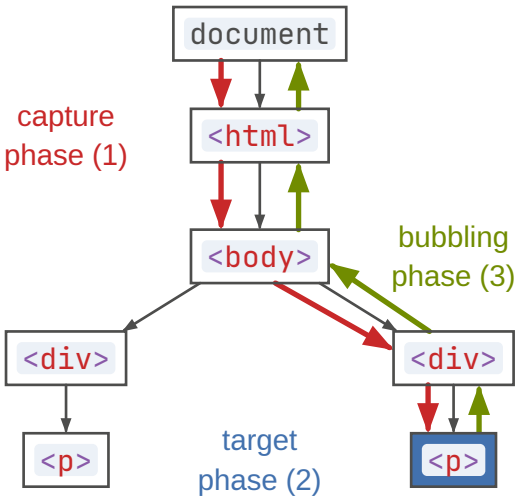
What happens when you add an event listener to a parent element, and the user clicks the child element.

e.target: refers to the element on which the event was initially fired

e.currentTarget: refers to the element to which this event handler has been attached

Events have 3 stages

1) Capture-Phase
   – Event walks DOM-tree from root to leaf
   – Each element can respond (but doesn't have to)
2) Target-Phase
   – Event reaches target element
   – All event listeners attached to the target element fire here
3) Bubbling-Phase
   – Event walks DOM-tree from leaf to root
   – Each element can respond (but doesn't have to)
   – Not all events do the bubbling phase



TODO:
`data-` attributes

5. CLEAN CODE

Intuitively understandable: Can be correctly understood with little effort and in a short time.

5.1. NAMING

Basic rule: What is my motivation to create this?

1) Names should be short, intuitive, and meaningful
2) Names should be pronounceable

– camelCase
– Constants: ALL_CAPS
– Boolean values: is…

– Arrays: Plural
– Familiar and specific words (`erase -> delete`)
– Consistency
– Do not repeat context (`todoService.addTodo -> todoService.add`)

`verbAttributeObject`: `removeEvery2ndChildElement(...)`

5.2. CODE QUALITY

Building blocks of software quality

Customer-oriented	*Manufacturer-oriented*
Functionality	Portability
Reliability	Modifiability
Efficiency	Testability
Usability	Transparency

DRY: Don’t Repeat Yourself

DIY: Do It Yourself

DIE: Release me from my misery

5.3. JS STYLEGUIDE (AIRBNB)

no-var: do not use `var`

prefer-const: use `let` only for variables with dynamic assignment

quotes: always use single quotes for strings

no-param-reassign: do not modify parameters inside a function

operator-linebreak: No line break directly before or after an assignment (`=`)

eqeqeq: use `===` and `!==` (not `==, !=`)

space-infix-ops: spaces around infix operators (e.g. not `const x=y+5`)

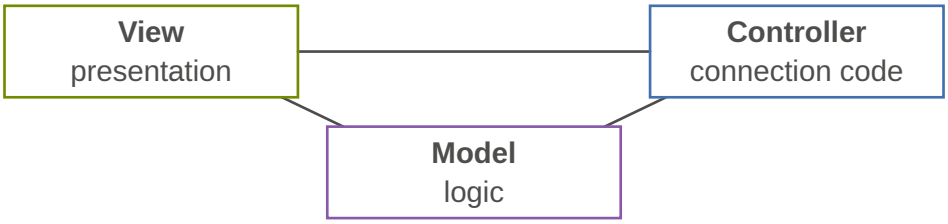
eol-last: A return at the end of the file

no-multiple-empty-lines: only one blank line

semi: end lines with a semicolon

Variable interpolation in strings should be done with `` `${}` ``, variables should preferably be written as individual statements on new lines instead of being comma-separated.

5.4. MVC



Model: Manages and changes state

- Data model
- App state
- Logic
- **No view refs**

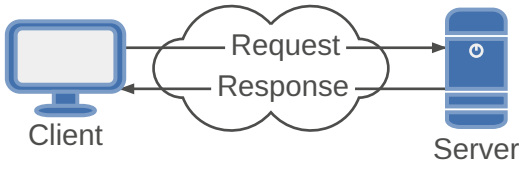
View: Displays elements in the DOM

Controller: Connects View & Model
- User input
- View changes (routing)

6. HYPERTEXT TRANSFER PROTOCOL (HTTP)

HTTP is stateless - client and server “forget” each other after every transaction. Each request and response contains all data.

Any kind of data can be transmitted over HTTP.



6.1. URL SCHEME

```
https://unterricht.ost.ch:443/stundenplan?org=I&sem=HS30#08:00
protocol://host:port/path?query#anchor
```

URL encoding turns any other character into a safe code the URL can carry. Each such character becomes a % sign followed by two hexadecimal digits. A space, for example, becomes %20. Special Characters must be encoded:

```
encodeURI("what's up? #gfeller ")          // "what's%20up?%20#gfeller%20"
encodeURIComponent("what's up? #gfeller") // "what's%20up%3F%20%23gfeller"
```

6.2. REQUEST

TODO:
W9A S12

Method Address Version Headers Header/body separator Request body

```
GET /index.php HTTP/3<crlf>
Host: www.html-world.de<crlf>
User-Agent: Mozilla/4.0<crlf>
Accept: image/gif, image/jpeg, */*<crlf>
Connection: close
<crlf>
<crlf>
...
```

Method	*Description*
GET	The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data.
HEAD	Same as GET, but transfers the status line and header section only.

POST	A POST request is used to send data to the server, for example, new customer information, file upload, etc. using HTML forms.
PUT	Replaces all current representations of the target resource with the uploaded content.
DELETE	Removes all current representations of the target resource given by a URI.
CONNECT	Establishes a tunnel to the server identified by a given URI.
OPTION	Describes the communication options for the target resource.
TRACE	Performs a message loop-back test along the path to the target resource.

GET data is placed into the URL (as query string), whereas POST data is placed into the request body.

6.3. RESPONSE

Version Status code Headers Header/body separator Request body

```
HTTP/2 200 OK <crlf>
Transfer-Encoding: chunked<crlf>
Date: Sat, 28 Nov 2030 04:36:25 GMT<crlf>
Pragma: public<crlf>
Etag: "pub1259380237;gz"<crlf>
Cache-Control: max-age=3600, public<crlf>
Content-Type: text/html; charset=UTF-8<crlf>
Last-Modified: Sat, 28 Nov 2030 03:50:37 GMT
<crlf>
<crlf>
<title>Title of page</title>
<!— ... rest of the html ... —>
```

6.3.1. Status codes

Code	*Description*	*Examples*
1xx	Informational	100 Continue
2xx	Successful	200 OK 201 Created 204 No Content
3xx	Redirection	301 Moved Permanently 304 Not Modified
4xx	Client Error	400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 418 I'm a teapot
5xx	Server Error	500 Internal Server Error 505 HTTP Version Not Supported
9xx	Non-Standard Codes	

6.3.2. Content-Type

Declares the parsing / interpreting format for the Client/Server. The Content-Type is declared in MIME-Type format

Type	*Description*	*Examples*
text	Represents any document that contains text and is theoretically human readable	text/plain text/html text/css text/typst
image	Represents any kind of images. Videos are not included, though animated images (like animated gif) are described with an image type.	image/gif image/png image/jpeg image/bmp image/webp
audio	Represents any kind of audio files	audio/midi audio/webm audio/ogg audio/wav
video	Represents any kind of video files	video/webm video/ogg
application	Represents any kind of binary data.	application/javascript application/octet-stream application/pkcs12 application/json application/xml application/pdf

6.3.3. Host

TODO:
W9A S23..

7. ASYNCHRONOUS JAVASCRIPT AND XML (AJAX)

With AJAX, web applications can send data to and retrieve from a server asynchronously without interfering with the display and behavior of the existing page.

7.1. ASYNC JS

Callback: A function passed into another function as an argument, which is then invoked inside the outer function to complete some kind of routine or action. (HOF)

Promise: A js object that represents the eventual completion (or failure) of an asynchronous operation. It can be in one of three states: pending, fulfilled (`resolve`), or rejected (`reject`).

Synchronous: Main thread is blocked while waiting for response

Asynchronous: Main thread isn't blocked, no "waiting" happens per se. Callbacks in promises or async/await can be used to write async js.

Example 9: **Promises**

```
function createTimer(time) {
  const startTime = performance.now()
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const timePassed = performance.now() - startTime;
      const inTime = Math.abs(time - timePassed) < 100;
      if (inTime) resolve();
      else reject("call to late");
    }, time)
  });
}

createTimer(1000)
  .then(snooze)
  .then(snooze)
  .then(snooze)
  .catch(console.error)
  .then(wakeUpCall)
  .finally(() => console.log("finished"))
```

Using `await` the function is paused, until the promise completes. The function must be marked `async`.

***Example** 10:*

```
try {
  await createTimer(1000)
  wakeUpCall();
} catch(e) {
  console.error(e);
}
```

⇔

```
createTimer(1000)
  .then(wakeUpCall)
  .catch(console.error)
```

7.2. FETCH

The Fetch API provides a JavaScript interface for making HTTP requests and processing the responses.

***Example** 11:*

```
try {
  const res = await fetch('/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: 'admin@admin.ch', pwd: '123456' })
  });
  console.log(res);   // { ..., ok: true, status: 200 }
```

```
  const data = await res.json();
  console.log(data);  // { ... }
} catch (err) {
  console.error(err); // { ..., ok: false, status: 400 }
}
```

The Response interface provides a number of methods to retrieve the entire body contents in a variety of different formats:

```
Response.json()
Response.arrayBuffer()
Response.blob()
Response.formData()
Response.text()
```